Role-based approval matrix & RBAC: map department drafts, Academic Affairs sign-off, and final registrar publish permissions

Computer
Registrar state machine & validation hooks: map draft, review, and publish transitions with immutable FERPA audit trails
Computer
Show the database schema for this approval matrix
Draft the API endpoints for state transitions
Show the event-driven notification architecture

Below is a unified spec for Semester's **registrar approval matrix, workflow schema, transition APIs, immutable audit model, and event-driven notification architecture.** Modern academic scheduling systems are designed to let departments build drafts, route proposals and exceptions through workflow, validate against institutional rules, and publish only after central approval and coordination.[1] [2] [3]

RBAC approval matrix

The permission model should enforce least privilege by role, scope, and action type, with separate rights for viewing, editing, approving, exporting, and overriding. FERPA-oriented privacy guidance and education RBAC guidance both emphasize granular access and comprehensive logging for sensitive records.[4] [5] [6]

Roles

Role	Scope	Core function
Department scheduler	Department/program	Build and submit schedule drafts
Department chair	Department/program	Review academic fit and staffing
Dean / associate dean	College/school	College-level approval and prioritization
Academic Affairs reviewer	Institution/college	Review policy-sensitive exceptions
Registrar scheduler	Institution/campus	Central validation, routing, and review
Registrar admin	Institution/campus	Final publish, lock, override authority
Room manager	Room pool / campus	Classroom assignment and room exceptions
Auditor / FERPA reviewer	Scoped read-only	Compliance, disclosure, and audit review

Approval matrix

Capability	Dept scheduler	Chair	Dean	Academic Affairs	Registrar scheduler	Registrar admin	Room manager	Auditor
View department drafts	Yes	Yes	Yes	Scoped	Yes	Yes	Scoped	Scoped
Edit draft record	Yes	No	No	No	Limited	Yes	No	No
Submit for workflow	Yes	Yes	Yes	No	Yes	Yes	No	No
Approve assigned step	No	Yes	Yes	Yes	Yes	Yes	No	No
Return for edits	No	Yes	Yes	Yes	Yes	Yes	Yes, room issues only	No

Capability	Dept scheduler	Chair	Dean	Academic Affairs	Registrar scheduler	Registrar admin	Room manager	Auditor
Route to Academic Affairs	Limited	Yes	Yes	No	Yes	Yes	No	No
Deny exception	No	No	Limited	Yes	Limited	Yes	No	No
Assign department room	Scoped	No	No	No	Yes	Yes	Yes	No
Assign central room	No	No	No	No	Yes	Yes	Yes	No
Override blocking conflict	No	No	No	Limited	Limited	Yes	Limited	No
Publish official schedule	No	No	No	No	No	Yes	No	No
Lock published schedule	No	No	No	No	Limited	Yes	No	No
Export scoped records	Limited	Limited	Limited	Limited	Yes	Yes	Limited	Limited
View audit log	No	No	Limited	Limited	Limited	Yes	No	Yes
View FERPA disclosure history	No	No	No	Limited	Limited	Yes	No	Yes

State-sensitive permissions

Workflow state	Dept scheduler	Chair	Dean	Academic Affairs	Registrar scheduler	Registrar admin
`draft`	Edit/save/submit	View	View	View	View	View
`validation_pending`	View	View	View	View	View	View
`validation_failed`	Edit/resubmit	View/comment	View/comment	View	View/comment	View/comment
`department_ready`	View	Approve/return	View	View	View	View
`chair_review`	View/comment	Approve/return	View	View	View	View
`dean_review`	View	Comment	Approve/return	View	View	View
`registrar_review`	View	View	View	View	Approve/return/route	Override
`room_coordination`	View/comment	View	View	View	Assign/return	Override
`academic_affairs_review`	View	View	View	Approve/deny/return	Comment	Override
`approved_for_publish`	View	View	View	View	View	Publish/return
`published`	Request change	Request change	Request change	View	Request change	Lock/unlock policy

Workflow state machine

Workflow engines in academic scheduling commonly route proposals to the correct approver, allow exception paths, and preserve step-by-step approval history. [2] [1]

States

```
draft
validation_pending
validation_failed
department_ready
chair_review
dean_review
```

```
registrar_review
room_coordination
academic_affairs_review
approved_for_publish
published
change_requested
locked
denied
archived
```

Transition table

From	Action	To	Condition
`draft`	Submit	`validation_pending`	Required fields present
`validation_pending`	Validation passed	`department_ready`	No blocking conflicts
`validation_pending`	Validation failed	`validation_failed`	Blocking rule violation
`validation_failed`	Resubmit	`validation_pending`	Errors corrected
`validation_failed`	Request exception	`academic_affairs_review`	Exception-eligible case
`department_ready`	Start chair review	`chair_review`	Department package complete
`chair_review`	Approve	`dean_review`	Chair decision recorded
`chair_review`	Return	`draft`	Comments required
`dean_review`	Approve	`registrar_review`	Dean decision recorded
`dean_review`	Route exception	`academic_affairs_review`	Policy-sensitive case
`dean_review`	Return	`draft`	Reason required
`registrar_review`	Approve	`room_coordination`	Central room step required
`registrar_review`	Approve	`approved_for_publish`	No room coordination needed
`registrar_review`	Route exception	`academic_affairs_review`	Registrar exception path
`registrar_review`	Return	`draft`	Correction required
`room_coordination`	Assign and clear	`approved_for_publish`	Room/resource issues resolved
`room_coordination`	Return	`registrar_review` or `draft`	Still unresolved
`academic_affairs_review`	Approve	`registrar_review`	Exception approved
`academic_affairs_review`	Deny	`denied`	Written reason required
`academic_affairs_review`	Return	`draft`	Rework required
`approved_for_publish`	Publish	`published`	Snapshot + audit event created
`published`	Request change	`change_requested`	Post-publish change flow
`change_requested`	Reopen	`validation_pending` or `registrar_review`	Depends on change severity
`published`	Lock	`locked`	Final term lock action
`locked`	Emergency unlock	`registrar_review`	Registrar admin only
`denied`	Rework	`draft`	New submission cycle
any	Archive	`archived`	Retention lifecycle

Validation hooks

Validation should trigger at:

- Draft save, as warnings.
- Submission, as blocking validation.
- Approval transitions, as stale-state rechecks.
- Pre-publish certification, as final enforcement.
- Nightly scans, as anomaly or drift detection.

Validation categories should include:

- Required data completeness.
- Meeting-pattern conformity.
- Instructor overlaps.
- Room overlaps.
- Room-capacity mismatch.
- Contact-hour mismatch.
- Cross-list consistency.
- Missing approvals.
- Publishability and lock-state checks.

Scheduling platforms and registrar workflows commonly rely on rule-based validation and approval routing to catch issues before publication.[1] [2]

Database schema

The schema should separate **academic data**, **workflow**, **RBAC**, **validation**, **notifications**, and **audit/FERPA** into distinct domains so approvals remain traceable and records stay governable.

Academic and workflow schema

```
CREATE TABLE terms (
  id uuid PRIMARY KEY,
  code text UNIQUE NOT NULL,
  name text NOT NULL,
  starts_on date NOT NULL,
  ends_on date NOT NULL,
  state text NOT NULL CHECK (
    state IN (
      'draft','validation_pending','validation_failed','department_ready',
      'chair_review','dean_review','registrar_review','room_coordination',
      'academic_affairs_review','approved_for_publish','published',
      'change_requested','locked','denied','archived'
    )
  ),
  publish_version integer NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE courses (
  id uuid PRIMARY KEY,
  subject_code text NOT NULL,
  catalog_number text NOT NULL,
  title text NOT NULL,
  credit_hours numeric(4,2) NOT NULL,
  owning_org_unit_id uuid NOT NULL,
  UNIQUE(subject_code, catalog_number)
);
```

```
CREATE TABLE sections (
  id uuid PRIMARY KEY,
  course_id uuid NOT NULL REFERENCES courses(id),
  term_id uuid NOT NULL REFERENCES terms(id),
  section_number text NOT NULL,
  owning_org_unit_id uuid NOT NULL,
  capacity integer NOT NULL CHECK (capacity > 0),
  workflow_state text NOT NULL CHECK (
    workflow_state IN (
      'draft','validation_pending','validation_failed','department_ready',
      'chair_review','dean_review','registrar_review','room_coordination',
      'academic_affairs_review','approved_for_publish','published',
      'change_requested','locked','denied','archived'
    )
  ),
  exception_flag boolean NOT NULL DEFAULT false,
  last_submitted_at timestamptz,
  published_at timestamptz,
  locked_at timestamptz,
  created_by uuid NOT NULL,
  updated_by uuid NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE(course_id, term_id, section_number)
);

CREATE TABLE rooms (
  id uuid PRIMARY KEY,
  building_code text NOT NULL,
  room_code text NOT NULL,
  capacity integer NOT NULL CHECK (capacity > 0),
  room_pool_type text NOT NULL CHECK (room_pool_type IN ('central','department')),
  owning_org_unit_id uuid,
  UNIQUE(building_code, room_code)
);

CREATE TABLE section_meetings (
  id uuid PRIMARY KEY,
  section_id uuid NOT NULL REFERENCES sections(id),
  day_of_week smallint NOT NULL CHECK (day_of_week BETWEEN 1 AND 7),
  start_time time NOT NULL,
  end_time time NOT NULL,
  room_id uuid REFERENCES rooms(id),
  instructor_id uuid,
  date_start date NOT NULL,
  date_end date NOT NULL
);

CREATE TABLE workflow_instances (
  id uuid PRIMARY KEY,
  entity_type text NOT NULL CHECK (entity_type IN ('section','term_calendar_event')),
  entity_id uuid NOT NULL,
  current_state text NOT NULL,
  initiated_by uuid NOT NULL,
  started_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE workflow_tasks (
  id uuid PRIMARY KEY,
  workflow_instance_id uuid NOT NULL REFERENCES workflow_instances(id),
  assigned_role_code text NOT NULL,
  assigned_scope_type text NOT NULL,
  assigned_scope_id uuid,
  assigned_user_id uuid,
  task_type text NOT NULL,
  status text NOT NULL CHECK (status IN ('open','approved','returned','denied','cancelled')),
  due_at timestamptz,
```

```
  created_at timestamptz NOT NULL DEFAULT now(),
  resolved_at timestamptz
);

CREATE TABLE workflow_transitions (
  id uuid PRIMARY KEY,
  workflow_instance_id uuid NOT NULL REFERENCES workflow_instances(id),
  from_state text NOT NULL,
  to_state text NOT NULL,
  action text NOT NULL,
  acted_by uuid NOT NULL,
  acted_as_role_code text NOT NULL,
  justification text,
  metadata jsonb NOT NULL DEFAULT '{}'::jsonb,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

Validation and conflict schema

```
CREATE TABLE validation_runs (
  id uuid PRIMARY KEY,
  entity_type text NOT NULL CHECK (entity_type IN ('section','term')),
  entity_id uuid NOT NULL,
  trigger_type text NOT NULL CHECK (trigger_type IN ('save','submit','transition','prepublish','batch')
  status text NOT NULL CHECK (status IN ('passed','failed','warning_only')),
  blocking_count integer NOT NULL DEFAULT 0,
  warning_count integer NOT NULL DEFAULT 0,
  created_by uuid,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE scheduling_conflicts (
  id uuid PRIMARY KEY,
  term_id uuid NOT NULL REFERENCES terms(id),
  conflict_type text NOT NULL CHECK (
    conflict_type IN (
      'instructor_overlap','room_overlap','capacity_mismatch',
      'pattern_violation','contact_hour_mismatch','feature_mismatch',
      'cross_list_mismatch'
    )
  ),
  severity text NOT NULL CHECK (severity IN ('warning','blocking')),
  left_section_id uuid NOT NULL REFERENCES sections(id),
  right_section_id uuid REFERENCES sections(id),
  room_id uuid,
  instructor_id uuid,
  status text NOT NULL CHECK (status IN ('open','waived','resolved')),
  opened_by_run_id uuid REFERENCES validation_runs(id),
  waiver_reason text,
  waived_by uuid,
  resolved_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

RBAC schema

```
CREATE TABLE roles (
  id uuid PRIMARY KEY,
  code text UNIQUE NOT NULL,
  name text NOT NULL
);

CREATE TABLE permissions (
  id uuid PRIMARY KEY,
  resource text NOT NULL,
```

```
  action text NOT NULL,
  UNIQUE(resource, action)
);

CREATE TABLE role_permissions (
  role_id uuid NOT NULL REFERENCES roles(id),
  permission_id uuid NOT NULL REFERENCES permissions(id),
  PRIMARY KEY (role_id, permission_id)
);

CREATE TABLE role_bindings (
  id uuid PRIMARY KEY,
  user_id uuid NOT NULL,
  role_id uuid NOT NULL REFERENCES roles(id),
  scope_type text NOT NULL CHECK (
    scope_type IN ('institution','campus','college','department','program','room_pool')
  ),
  scope_id uuid NOT NULL,
  starts_at timestamptz,
  ends_at timestamptz,
  granted_by uuid NOT NULL,
  justification text,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

Audit, FERPA, and notification schema

```
CREATE TABLE audit_events (
  id uuid PRIMARY KEY,
  actor_user_id uuid NOT NULL,
  acted_as_role_code text NOT NULL,
  module text NOT NULL,
  action text NOT NULL,
  target_type text NOT NULL,
  target_id uuid NOT NULL,
  scope_type text NOT NULL,
  scope_id uuid NOT NULL,
  reason_code text,
  legitimate_interest text,
  metadata jsonb NOT NULL DEFAULT '{}'::jsonb,
  ip_address inet,
  user_agent text,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE ferpa_disclosure_requests (
  id uuid PRIMARY KEY,
  student_id uuid NOT NULL,
  requester_type text NOT NULL,
  requester_identity text NOT NULL,
  requested_record_type text NOT NULL,
  purpose text NOT NULL,
  status text NOT NULL CHECK (status IN ('submitted','under_review','approved','denied','fulfilled')),
  reviewed_by uuid,
  decision_reason text,
  created_at timestamptz NOT NULL DEFAULT now(),
  decided_at timestamptz,
  fulfilled_at timestamptz
);

CREATE TABLE notification_subscriptions (
  id uuid PRIMARY KEY,
  user_id uuid NOT NULL,
  event_type text NOT NULL,
  channel text NOT NULL CHECK (channel IN ('in_app','email','sms','webhook')),
  enabled boolean NOT NULL DEFAULT true,
  scope_type text,
```

```
  scope_id uuid,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE notification_events (
  id uuid PRIMARY KEY,
  event_type text NOT NULL,
  aggregate_type text NOT NULL,
  aggregate_id uuid NOT NULL,
  payload jsonb NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE notification_deliveries (
  id uuid PRIMARY KEY,
  notification_event_id uuid NOT NULL REFERENCES notification_events(id),
  recipient_user_id uuid NOT NULL,
  channel text NOT NULL CHECK (channel IN ('in_app','email','sms','webhook')),
  status text NOT NULL CHECK (status IN ('queued','sent','failed','suppressed','read')),
  template_key text,
  sent_at timestamptz,
  failed_at timestamptz,
  error_code text
);
```

API endpoints for state transitions

Workflow endpoints should be explicit transition commands rather than generic status updates, because that preserves policy checks and auditable intent. Workflow systems in academic scheduling are commonly built around routed requests and approver decisions. [2] [1]

Section lifecycle endpoints

```
GET    /api/terms/{termId}/sections
POST   /api/terms/{termId}/sections
GET    /api/sections/{sectionId}
PATCH  /api/sections/{sectionId}
POST   /api/sections/{sectionId}/submit
POST   /api/sections/{sectionId}/resubmit
POST   /api/sections/{sectionId}/request-exception
POST   /api/sections/{sectionId}/request-change
POST   /api/sections/{sectionId}/lock
POST   /api/sections/{sectionId}/unlock-emergency
```

Workflow decision endpoints

```
GET    /api/workflow/tasks?assignee=me
GET    /api/workflow/instances/{instanceId}
POST   /api/workflow/tasks/{taskId}/approve
POST   /api/workflow/tasks/{taskId}/return
POST   /api/workflow/tasks/{taskId}/deny
POST   /api/workflow/tasks/{taskId}/route-academic-affairs
POST   /api/workflow/tasks/{taskId}/comment
```

Validation and rooming endpoints

```
POST   /api/sections/{sectionId}/validate
GET    /api/sections/{sectionId}/conflicts
GET    /api/conflicts?termId=&orgUnitId=&severity=&status=
POST   /api/conflicts/{conflictId}/waive
POST   /api/conflicts/{conflictId}/resolve
```

```
GET    /api/rooms/availability?termId=&day=&start=&end=&capacity=
POST   /api/sections/{sectionId}/assign-room
```

Publish and audit endpoints

```
POST   /api/terms/{termId}/prepublish-check
GET    /api/terms/{termId}/publication-preview
POST   /api/terms/{termId}/publish
POST   /api/terms/{termId}/enter-refine-mode
POST   /api/terms/{termId}/final-lock

GET    /api/audit-events?targetType=section&targetId=
GET    /api/audit-events?actorId=&module=registrar
POST   /api/exports/sections
POST   /api/ferpa/disclosure-requests
POST   /api/ferpa/disclosure-requests/{id}/decide
```

Transition-to-event examples

Endpoint	Domain event emitted
`POST /api/sections/{id}/submit`	`section.submitted`
`POST /api/workflow/tasks/{id}/approve`	`workflow.task_approved`
`POST /api/workflow/tasks/{id}/route-academic-affairs`	`section.exception_routed`
`POST /api/sections/{id}/assign-room`	`section.room_assigned`
`POST /api/terms/{id}/publish`	`term.schedule_published`
`POST /api/exports/sections`	`records.export_requested`

Event-driven notification architecture

Event-driven architecture fits institutional workflows because approvals, assignments, exceptions, and publications are all discrete business events that should trigger notifications, integrations, and audit records without tightly coupling every module. Higher-ed event-driven architecture guidance explicitly frames it as a way to coordinate workflows, rules, and audit-preserving automation across departments and systems. [7]

Architecture flow

```
User action / system validation
  → Workflow transition committed
  → Domain event written
  → Event bus / queue
  → Notification policy engine
  → Channel fan-out
  → Delivery tracking
  → Read / failure / retry events
```

Core components

Component	Responsibility
Workflow service	Commits state change and transition record
Outbox/event table	Guarantees event publication after commit
Event bus	Delivers domain events to subscribers
Notification policy engine	Decides recipients, channels, quiet hours, suppression
Template service	Renders message content

Component	Responsibility
Delivery workers	Send in-app, email, SMS, or webhook messages
Audit service	Logs sensitive notification and disclosure actions
Preference service	Applies opt-in and channel settings

Example events

Event	Trigger	Typical recipients
`section.submitted`	Department submits section	Chair, dean queue owners
`workflow.task_assigned`	New review task opened	Assigned approver
`workflow.task_returned`	Section returned for edits	Department scheduler
`section.exception_routed`	Exception sent to Academic Affairs	Academic Affairs reviewers, registrar
`section.room_conflict_detected`	Blocking room conflict found	Department scheduler, registrar room team
`term.schedule_published`	Official publish completed	Department schedulers, downstream integrations
`records.export_requested`	Sensitive export requested	Privacy/compliance reviewer, requester
`ferpa.disclosure_decided`	Disclosure request approved/denied	Compliance team, request owner

Notification rules

Use a policy engine to determine:

- Who should be notified by role and scope.
- Which channels are allowed by event severity.
- When to suppress duplicates.
- When escalation reminders fire.
- When a compliance event also requires audit review.

Examples:

- Immediate in-app + email for workflow task assignment.
- Escalation email if an approval task nears SLA breach.
- No SMS for routine returns unless the institution enables it.
- Special compliance routing for exports or FERPA disclosure events.

Reliability model

Use an outbox pattern so the workflow transition and the emitted event are committed together, then delivered asynchronously. Event-driven architecture guidance for higher education highlights the need for defined ownership, state preservation, policy control, and measurable outcomes across workflows.[7]

Delivery tracking

Every notification should track:

- Event ID.
- Recipient.
- Channel.
- Template used.
- Delivery status.

- Retry count.
- Suppression reason.
- Read/open status where applicable.

Sensitive notifications, especially those about disclosures, exports, or access issues, should also produce audit events because they relate to regulated record handling. FERPA-oriented tooling guidance stresses full lifecycle logging, including access, exports, and disclosure-related actions. [5] [8] [4]

FERPA audit requirements

Immutable audit trails are a core requirement. FERPA-focused compliance guidance repeatedly emphasizes logging each access, modification, export, and disclosure event, and preserving those records in tamper-resistant form. [9] [8] [4]

Must-log events

- Record access to FERPA-relevant student-linked schedule data.
- Workflow approval, return, deny, and reroute actions.
- Role grants and revocations.
- Sensitive exports.
- Publish, lock, and emergency unlock.
- Disclosure request intake, decision, and fulfillment.
- Failed authorization attempts on restricted records.

Minimum audit payload

Field	Description
Actor ID	User or service account
Effective role	Role used for authorization
Scope	Campus/college/department/room pool
Action	View, edit, approve, export, publish, disclose
Target	Section, term, record, disclosure request
Timestamp	When action occurred
Legitimate interest	Reason access was allowed
Client metadata	IP, user agent, request ID
Outcome	Success, denied, returned, failed

FERPA-specific controls

- Append-only audit storage.
- Separate disclosure log from ordinary access log.
- Export review and justification workflow.
- Periodic access review by scope and role.
- Centralized reporting for investigations and parent/student requests.
- Retention long enough to support audits and incident reviews, with institution-defined policy. Sources recommend centralized logging and long-term retention for investigation readiness. [8]

Recommended implementation pattern

The cleanest design is:

1. **RBAC policy engine** resolves action rights by role and scope.
2. **Workflow service** enforces state transitions and validation hooks.
3. **Outbox/event bus** emits domain events after successful commits.
4. **Notification engine** handles fan-out, retries, and preferences.
5. **Audit/FERPA service** records immutable evidence for access, exports, and disclosures.

That gives Semester a registrar-grade approval console with department drafting, Academic Affairs exception routing, final publish control, and a defensible compliance trail. [4] [1] [2]

⁂

1. https://coursedog.my.site.com/support/s/article/WORKFLOWSWorkflowBasics67fd8e827574b
2. https://www.courseleaf.com/software/clss/
3. https://registrar.uccs.edu/facultystaff/academic-scheduling/the-scheduling-process
4. https://openeducat.org/features/ferpa-compliance/
5. https://www.fullfabric.com/articles/student-data-privacy-ferpa-compliance-higher-education
6. https://www.illumio.com/cybersecurity-101/rbac
7. https://www.creatrixcampus.com/higher-education-glossary/event-driven-architecture
8. https://secureprivacy.ai/blog/ferpa-compliance-software
9. https://www.codetrade.io/blog/ferpa-compliant-lms-us-universities/
10. https://registrar.vanderbilt.edu/faculty-staff/academic-scheduling.php
11. https://www.vectorsolutions.com/resources/blogs/ferpa-compliance-higher-education/